
SENTIMENT ANALYSIS SERVICE

Sổ tay kỹ thuật và báo cáo phân quyền sở hữu

Phục vụ CTO review, due diligence, readiness, handover, onboarding

Tech Lead	Le Quang Tuyen
ML Engineering Lead	Duong Binh An
AI Engineering Lead	Do Quoc Trung
Solution Engineering Lead	Duong Hong Quan
Backend Engineering Lead	Pham Duc Long
UI/UX Engineering Lead	Nguyen Le Hong Nhi

Thứ tự nguồn sự thật dùng trong tài liệu này:

1. Mã nguồn
 2. Cấu hình runtime
 3. Manifest hạ tầng
 4. Kiểm thử
 5. Báo cáo cũ
 6. README và tài liệu hướng dẫn
-
-

Contents

1	Phần 1 - Đánh giá điều hành	3
1.1	Bài toán kinh doanh	3
1.2	Mục đích sản phẩm	3
1.3	Mục tiêu kỹ thuật	3
1.4	Người dùng cuối	3
1.5	Giá trị cốt lõi	3
1.6	Phân loại mức trưởng thành	4
1.7	Lý do xếp hạng theo implementation	4
2	Phần 2 - Mô hình tư duy hệ thống	4
2.1	Mô hình tư duy kinh doanh	4
2.2	Mô hình tư duy sản phẩm	4
2.3	Mô hình tư duy kỹ thuật	5
2.4	Mô hình tư duy vận hành	5
2.5	Mô hình tư duy AI/ML	5
2.6	Giải thích ngắn gọn cho người mới	6
3	Phần 3 - Luồng end-to-end đầy đủ	6
3.1	Bảng luồng	6
3.2	Chuỗi runtime chính	7
3.3	Chuỗi training đến serving	8
4	Phần 4 - Reverse engineering repository	8
4.1	Phân tích thư mục chính	8
4.2	Đồ thị phụ thuộc thư mục	9
4.3	Đồ thị phụ thuộc runtime	10
4.4	Đồ thị phụ thuộc training	10
4.5	Đồ thị phụ thuộc deployment	10
5	Phần 5 - Phân tích file quan trọng theo mức độ	10
5.1	Top 50 file quan trọng nhất	10
5.2	Ghi chú mức độ quan trọng	18
6	Phần 6 - Trace thực thi	18
6.1	Khởi động ứng dụng	18
6.2	Health Check	19
6.3	Predict	19
6.4	Explain	19
6.5	Batch Predict	19
6.6	Evaluation	19
6.7	Training	20
6.8	Export ONNX	20

6.9	Sequence diagram	20
7	Phần 7 - Sổ tay Frontend	20
7.1	Owner	20
7.2	Kiến trúc	20
7.3	Components	20
7.4	Services và state flow	21
7.5	Explainability UI	21
7.6	Batch Upload UI	21
7.7	Rủi ro chính	21
7.8	Điểm mở rộng	21
8	Phần 8 - Sổ tay Backend	21
8.1	Owner	21
8.2	Cấu trúc FastAPI	21
8.3	Contracts và validation	22
8.4	Bản đồ ownership API	22
8.5	Xử lý lỗi	22
8.6	Khoảng trống backend	22
9	Phần 9 - Sổ tay AI inference	22
9.1	Owner	22
9.2	Pipeline	22
9.3	Tokenization và tensor	23
9.4	ONNX Runtime	23
9.5	Logic prediction	23
9.6	ABSA	23
9.7	SHAP	23
9.8	Phát hiện ngôn ngữ	23
9.9	Phân tích độ trễ	23
10	Phần 10 - Sổ tay ML	23
10.1	Owner	23
10.2	Chiến lược dữ liệu	24
10.3	DVC	24
10.4	MLflow	24
10.5	LoRA và PEFT	24
10.6	Đánh giá và fairness	24
10.7	Vòng đời training	24
10.8	Khoảng trống ML	24
11	Phần 11 - Sổ tay Solution Engineering	24
11.1	Owner	25
11.2	Tích hợp end-to-end	25

11.3 Kiến trúc triển khai	25
11.4 Chiến lược monitoring	25
11.5 Đánh giá readiness vận hành	25
12 Phần 12 - Ma trận ownership	25
12.1 Ma trận ownership thư mục	26
12.2 Ma trận ownership service	26
12.3 Operational, security, documentation, knowledge ownership	26
13 Phần 13 - Ma trận RACI	27
14 Phần 14 - Data và model lineage	27
15 Phần 15 - Security review	28
16 Phần 16 - Performance review	29
16.1 Startup Time	29
16.2 Memory Usage	29
16.3 CPU Usage	29
16.4 Inference Latency	29
16.5 Batch Throughput	29
16.6 SHAP Cost	29
16.7 ABSA Cost	29
16.8 Giới hạn mở rộng	29
17 Phần 17 - Sổ tay vận hành	30
18 Phần 18 - Sổ đăng ký technical debt	31
19 Phần 19 - Đánh giá tổ chức	32
19.1 Cấu trúc đội ngũ	32
19.2 Phân phối tri thức và bus factor	32
19.3 Khoảng trống ownership	33
19.4 Khuyến nghị	33
20 Phần 20 - Scorecard production readiness	33
21 Phần 21 - Roadmap	33
21.1 Roadmap 3 tháng	33
21.2 Roadmap 6 tháng	34
21.3 Roadmap 12 tháng	34
22 Phần 22 - Hướng dẫn onboarding	35
23 Phần 23 - CTO review	35
23.1 Có phê duyệt production launch không?	35

23.2 Có phê duyệt enterprise customers không?	35
23.3 Có phê duyệt scaling không?	36
23.4 Điều đáng lo nhất là gì?	36
23.5 Khoản đầu tư đầu tiên nên là gì?	36
Discrepancy implementation	36

1 Phần 1 - Đánh giá điều hành

1.1 Bài toán kinh doanh

Hệ thống giải bài toán chuyển phản hồi văn bản tự do thành tín hiệu vận hành có cấu trúc để phục vụ phân tích sản phẩm, triage hỗ trợ khách hàng, giám sát chất lượng dịch vụ, và trình diễn năng lực AI. Nếu làm thủ công, khối lượng đánh giá không mở rộng được và không tạo ra dữ liệu có thể hành động.

1.2 Mục đích sản phẩm

Repository hiện cung cấp:

- giao diện Angular dạng chat và batch upload cho người dùng cuối
- dịch vụ FastAPI cho predict, explain, health, metrics, batch predict và evaluate
- pipeline ML ngoại tuyến cho download dữ liệu, tiền xử lý, validation, finetuning, đánh giá, và export ONNX

1.3 Mục tiêu kỹ thuật

Mục tiêu kỹ thuật là chứng minh một chuỗi ML-in-production hoàn chỉnh:

- DVC điều phối pipeline dữ liệu và huấn luyện
- MLflow theo dõi thí nghiệm và chỉ số
- PEFT/LoRA cho fine-tuning
- ONNX Runtime cho serving tối ưu
- Prometheus và Grafana cho quan sát vận hành
- Docker Compose để đóng gói hệ thống tích hợp

1.4 Người dùng cuối

- stakeholder sản phẩm muốn xem sentiment nhanh
- người dùng demo tương tác với chatbot có explainability
- kỹ sư vận hành backend, inference, training và monitoring
- chủ dự án tương lai cần tiếp quản toàn bộ vòng đời hệ thống

1.5 Giá trị cốt lõi

Giá trị chính không nằm ở việc có một mô hình sentiment đơn lẻ. Giá trị nằm ở việc đóng gói sentiment inference thành dịch vụ có API, explainability, batch scoring, monitoring, lineage mô hình, và quy trình training có thể lặp lại.

1.6 Phân loại mức trưởng thành

Mức	Đánh giá
Prototype	Đã vượt qua. Repo có API, Docker, test, DVC, MLflow, ONNX export, và dashboard monitoring.
MVP	Đã vượt qua. Có predict, explain, health, metrics, batch CSV, và UI cơ bản.
Advanced MVP	Trạng thái hiện tại. Hệ thống có luồng end-to-end thực, có monitoring, có training life-cycle, nhưng còn thiếu các kiểm soát production.
Pre-Production	Đạt một phần. Có container hóa và cấu trúc vận hành, nhưng thiếu auth, batch status là mock, nhiều state trong memory, và chưa có durable job system.
Production Ready	Chưa đạt. Phù hợp demo nội bộ hoặc pilot giới hạn; chưa phù hợp môi trường doanh nghiệp thực thụ.

1.7 Lý do xếp hạng theo implementation

- Runtime API chạy thật trong `src/main.py`, có startup load model, metrics, explain, batch, evaluate.
- UI Angular kết nối được tới API, nhưng contract chưa khớp hoàn toàn với backend.
- Pipeline training có thật qua `dvc.yaml`, `params.yaml`, `src/scripts/run_finetuning.py`, `export_onnx.py`.
- Monitoring stack tồn tại trong Compose với Prometheus, Grafana và alert rules.
- Các kiểm soát production quan trọng chưa có: authentication, authorization, secret hygiene tốt, state persistence, rate limiting, job queue bền vững.

2 Phần 2 - Mô hình tư duy hệ thống

2.1 Mô hình tư duy kinh doanh

```

1 flowchart LR
2     Feedback[Phản hồi khách hàng] --> Analysis[Phân tích sentiment]
3     Analysis --> Insight[Insight vận hành]
4     Insight --> Action[Uu tiên xử lý]
5     Action --> BetterService[Cải thiện sản phẩm và dịch vụ]
```

Hệ thống nhận câu đánh giá và trả ra kết quả sentiment có thể dùng để phân loại mức độ hài lòng, nhận diện chủ đề bị phàn nàn, và tạo cơ sở cho hành động tiếp theo.

2.2 Mô hình tư duy sản phẩm

```

1 flowchart LR
2     User[Người dùng] --> UI[Angular UI]
3     UI --> API[FastAPI]
4     API --> Model[Sentiment + ABSA + SHAP]
5     Model --> Result[Kết quả có giải thích]
```

```
6 Result --> User
```

Về góc nhìn sản phẩm, đây là một công cụ phân tích sentiment có hai chế độ chính: chat một câu và batch upload CSV. Trải nghiệm giải thích đóng vai trò tăng niềm tin hơn là tối ưu throughput.

2.3 Mô hình tư duy kỹ thuật

```
1 flowchart TD
2     subgraph Runtime
3         FE[Angular]
4         Nginx[Nginx]
5         API[FastAPI]
6         ORT[ONNX Runtime]
7     end
8     subgraph ML
9         Data[DVC stages]
10        Train[HF Trainer + PEFT]
11        Eval[Evaluation]
12        Export[ONNX Export]
13        Track[MLflow]
14    end
15    FE --> Nginx --> API --> ORT
16    Data --> Train --> Eval --> Export --> ORT
17    Train --> Track
18    Eval --> Track
```

Hệ thống chia rõ hai mặt: mặt online phục vụ inference và mặt offline phục vụ dữ liệu, training, đánh giá, export. Điểm nối giữa hai mặt là artifact ONNX và cấu hình model.

2.4 Mô hình tư duy vận hành

```
1 flowchart LR
2     Compose[Docker Compose] --> Services[API/UI/Prometheus/Grafana/MLflow]
3     Services --> Metrics[/metrics/]
4     Metrics --> Prometheus
5     Prometheus --> Grafana
6     Prometheus --> Alerts[Alert rules]
```

Về vận hành, hệ thống dựa vào Compose cho môi trường tích hợp. Monitoring quan sát được nhưng response playbook và độ bền trạng thái vẫn chưa hoàn chỉnh.

2.5 Mô hình tư duy AI/ML

```
1 flowchart LR
2     Raw[Raw data] --> Prep[Tiền xử lý]
3     Prep --> Finetune[Finetuning]
4     Finetune --> Evaluate[Đánh giá]
5     Evaluate --> MLflow[MLflow]
6     Finetune --> Export[Export ONNX]
7     Export --> Serve[Inference runtime]
```


Mô hình chính cho sentiment được fine-tune, sau đó merge adapter và export sang ONNX để phục vụ inference. ABSA và sarcasm là khả năng bổ trợ, không phải phần cứng cốt lõi của throughput.

2.6 Giải thích ngắn gọn cho người mới

Nếu nhìn ở mức cao, repository này là một chuỗi khép kín:

- dữ liệu được tải và làm sạch
- mô hình được fine-tune và đánh giá
- artifact ONNX được sinh ra cho runtime
- API nạp artifact và trả kết quả cho UI
- metrics được Prometheus thu thập và Grafana trực quan hóa

Điểm yếu lớn nhất hiện tại không phải là thiếu ý tưởng kiến trúc mà là thiếu các kiểm soát production và một vài chỗ contract drift giữa UI, docs, và backend.

3 Phần 3 - Luồng end-to-end đầy đủ

3.1 Bảng luồng

Bước	Mục tiêu	Implementation	Owner	Phụ thuộc	Failure mode
Người dùng	Gửi yêu cầu sentiment	Chat UI hoặc batch upload	UI/UX	Browser, frontend build	UI validation không chặn lỗi logic
Frontend	Thu thập input và gọi API	Angular service dùng HttpClient	UI/UX	Nginx proxy, contract model	lang không luôn được gửi, mismatch option
API gateway	Định tuyến request	Nginx proxy /api/ về backend	Solution	Container networking	Sai prefix gây 404 hoặc contract drift
Backend	Validate và điều phối inference	FastAPI route trong src/main.py	Backend	Pydantic, app state	Thiếu auth, state memory-only
Inference layer	Biến text thành logits và nhãn	ONNXInferenceModule và helpers	AI	Tokenizer, ONNX Runtime	Model load fail, tokenizer mismatch
Model	Tạo dự đoán	ONNX model hoặc baseline path	AI	Model files, config	Artifact lỗi hoặc version drift
Response	Trả kết quả có cấu trúc	Pydantic response models	Backend	JSON serialization	Response shape lệch docs/UI
Monitoring	Ghi metrics	Prometheus metrics endpoint	Solution	Prometheus scrape	Monitoring stack down
Feedback	Lưu phản hồi nghiệp vụ	Not Implemented	Solution	N/A	Không có closed-loop feedback
Training	Cập nhật mô hình	DVC + HF + PEFT + MLflow	ML	Datasets, GPU/CPU, params	Data drift, evaluation yếu

Deployment	Đưa artifact vào runtime	Docker build + Compose runtime	Solution	Image build, Thay model không mounted model kiểm soát rollout files
------------	--------------------------	--------------------------------	----------	---

3.2 Chuỗi runtime chính

```

1 sequenceDiagram
2     participant U as User
3     participant FE as Angular Frontend
4     participant NX as Nginx
5     participant API as FastAPI
6     participant INF as ONNXInferenceModel
7     participant MON as Prometheus
8
9     U->>FE: nhập văn bản hoặc tải CSV
10    FE->>NX: gọi /api/predict hoặc /api/batch_predict
11    NX->>API: forward request
12    API->>INF: tokenize và infer
13    INF-->>API: logits, probabilities, labels
14    API-->>NX: JSON response
15    NX-->>FE: kết quả
16    FE-->>U: hiển thị sentiment, confidence, explain
17    MON->>API: scrape /metrics

```

3.3 Chuỗi training đến serving

```

1 sequenceDiagram
2     participant D as DVC Pipeline
3     participant T as Finetuning Script
4     participant M as MLflow
5     participant E as Export ONNX
6     participant R as Runtime API
7
8     D->>T: dữ liệu đã chuẩn hóa
9     T->>M: log params và metrics
10    T-->>E: checkpoint tốt nhất
11    E-->>R: ONNX model + tokenizer config
12    R->>R: startup load artifact

```

4 Phần 4 - Reverse engineering repository

4.1 Phân tích thư mục chính

Thư mục	Mục đích	Liên quan runtime	Owner	Mức rủi ro
app/sentiment-analysis	Frontend Angular, assets, Nginx config	Cao	UI/UX	Trung bình đến cao vì contract drift tác động trực tiếp người dùng

src	Mã nguồn backend, inference, data, training scripts	Rất cao	Backend, AI, ML	Rất cao; đây là lõi hệ thống
contracts	API schema, contract examples	Trung bình	Backend + Solution	Trung bình; docs drift có thể làm đội tích hợp hiểu sai
infra	Prometheus, Grafana provisioning, dashboards	Cao	Solution	Cao nếu monitoring hỏng hoặc cấu hình sai
models	Artifact model phục vụ runtime hoặc export	Rất cao	AI + ML	Rất cao; mất artifact thì runtime không phục vụ được
data	Dữ liệu thô, đã xử lý, mẫu, eval inputs	Trung bình đến cao	ML	Cao với training, thấp hơn với runtime
tests	API, performance, benchmark tests	Trung bình	Backend + AI + ML	Trung bình; thiếu test làm tăng rủi ro thay đổi
docs	Tài liệu, handbook, review cũ	Thấp đối với runtime	Tech Lead + Solution	Trung bình vì có thể gây hiểu nhầm nếu lệch code
reports	Báo cáo phân tích trước đó	Thấp	Tech Lead	Thấp; dùng làm tham khảo
notebooks	Khám phá và thực nghiệm	Thấp đến trung bình	ML + AI	Trung bình; kiến thức có thể nằm ở đây nhưng không phải runtime chính
mlruns	Tracking local của MLflow	Thấp cho runtime, cao cho ML traceability	ML	Trung bình; mất history thì giảm auditability

4.2 Đồ thị phụ thuộc thư mục

```

1 flowchart TD
2   UI[app/sentiment-analysis-chatbot] --> SRC[src]
3   SRC --> MODELS[models]
4   SRC --> DATA[data]
5   SRC --> INFRA[infra]
6   TESTS[tests] --> SRC
7   CONTRACTS[contracts] --> SRC
8   DOCS[docs] --> SRC

```

4.3 Đồ thị phụ thuộc runtime

```

1 flowchart LR
2   Browser --> Angular
3   Angular --> Nginx
4   Nginx --> FastAPI
5   FastAPI --> ONNX
6   FastAPI --> Metrics

```

```

7 Metrics --> Prometheus
8 Prometheus --> Grafana

```

4.4 Đồ thị phụ thuộc training

```

1 flowchart LR
2   RawData --> Downloader
3   Downloader --> Pipeline
4   Pipeline --> Validators
5   Validators --> Finetuning
6   Finetuning --> Evaluate
7   Finetuning --> MLflow
8   Finetuning --> ONNXExport

```

4.5 Đồ thị phụ thuộc deployment

```

1 flowchart TD
2   Dockerfile --> APIImage
3   DockerfileTrain --> TrainImage
4   Compose --> APIImage
5   Compose --> FrontendImage
6   Compose --> Prometheus
7   Compose --> Grafana
8   Compose --> MLflow

```

5 Phần 5 - Phân tích file quan trọng theo mức độ

5.1 Top 50 file quan trọng nhất

Hạng	File	Vì sao tồn tại	Owner	Tác động nếu mất
1	src/main.py	Entry point runtime và toàn bộ API contract thật	Backend	API không chạy
2	src/model/onnx_inference.py	Pipeline serving ONNX	AI	Predict path hỏng
3	src/model/config.py	Cấu hình model, nhãn, ngôn ngữ hỗ trợ	AI	Runtime lệch model
4	src/model/baseline.py	Wrapper inference baseline	AI	Đường fallback suy giảm
5	src/model/onnx_exporter.py	Chuyển checkpoint thành artifact runtime	AI + ML	Không thể đóng vòng training sang serving
6	src/scripts/run_finetuning.py	Script huấn luyện chính	ML	Không vận hành được training
7	src/scripts/evaluate_finetuning_checkpoint.py	Đánh giá checkpoint	ML	Không chấm được chất lượng sau train

8	src/scripts/export_onnx.py	Entry point export ONNX	AI + ML	Không tạo được artifact runtime
9	src/data/pipeline.py	Tiền xử lý dữ liệu	ML	Dataset huấn luyện sai hoặc thiếu
10	src/data/validators.py	Kiểm soát chất lượng dữ liệu	ML	Data quality suy giảm
11	src/data/downloader.py	Tải dữ liệu nguồn	ML	Không tải lập được pipeline
12	src/training/dataset_builder.py	Tạo dataset cho trainer	ML	Train input không dựng được
13	src/training/metrics.py	Tính metric	ML	Báo cáo chất lượng sai
14	src/training/weighted_trainer.py	Trainer tùy biến xử lý imbalance	ML	Hiệu quả train giảm
15	src/training/task_config.py	Cấu hình task huấn luyện	ML	Lifecycle nhiều task bị lệch
16	src/training/lora_config.py	Cấu hình LoRA/PEFT	ML	Fine-tuning không nhất quán
17	src/training/mlflow_callback.py	Log ML flow trong training	ML	Mất traceability
18	src/model/evaluate.py	Runtime evaluate helper	AI + Backend	Endpoint evaluate hỏng
19	src/model/language_detector.py	Suy luận ngôn ngữ	AI	Routing ngôn ngữ giảm chính xác
20	src/model/device.py	Quyết định CPU/GPU/MPS	AI	Runtime chọn thiết bị sai
21	docker-compose.yml	Bản đồ triển khai tích hợp	Solution	Không dựng được stack
22	Dockerfile	Build ảnh backend	Solution	API container không build
23	Dockerfile.train	Build ảnh training	Solution + ML	Train image không build
24	app/.../src/app/app.component.ts	App shell frontend	UI/UX	Luồng chat UI gây
25	app/.../services/sentiment-analysis.service.ts	Gửi API từ UI	UI/UX	Frontend không giao tiếp backend
26	app/.../chat-input.component.ts	Nhập câu, chọn ngôn ngữ	UI/UX	Tương tác người dùng suy giảm
27	app/.../batch-upload.component.ts	Upload CSV batch	UI/UX	Batch UI mất chức năng
28	app/.../models/message.contract.ts	Contract hiển thị message	UI/UX	Rendering chat sai
29	app/sentiment-analysis-proxy	Proxy và phục vụ frontend	Solution	Tích hợp FE-BE lỗi
30	infra/prometheus/prometheus.yml	Scrape config	Solution	Metrics không thu thập
31	infra/prometheus/alertmanager.yml	Cảnh báo	Solution + SRE	Không có alert thực dụng
32	infra/grafana/provisioning/datasources/datasource.yml	Data source Grafana	Solution	Dashboard mù dữ liệu
33	infra/grafana/dashboards/dashboard.yml	Dashboard chính	Solution	Mất quan sát trực quan
34	tests/test_api.py	Bảng chứng contract API cơ bản	Backend	Regression dễ lọt
35	tests/perf/test_inference_latency.py	Guardrail latency	AI	Hiệu năng giảm không bị phát hiện

36	tests/model/test_batch_output	Guardrail through-put	AI	Batch path thoái hóa
37	dvc.yaml	Định nghĩa pipeline dữ liệu/train/eval	ML	Mất orchestration
38	params.yaml	Tham số pipeline/training	ML	Run không nhất quán
39	requirements.txt	Dependency run-time/train	Solution + Tech Lead	Build và run có thể hỏng
40	contracts/openapi.yaml	Hợp đồng giao tiếp mong muốn	Backend + Solution	Tích hợp bên ngoài lệch
41	contracts/README.md	Giải thích contract	Backend + Solution	Dễ tạo hiểu nhầm nếu drift
42	README.md	Điểm vào cho người mới	Tech Lead	Onboarding chậm và sai kỳ vọng
43	src/scripts/prepare_evaluation	Chuẩn bị tập đánh giá	ML	Đánh giá khó lặp lại
44	src/scripts/benchmark_on_inference	Benchmark tốc độ inference	AI	Thiếu dữ liệu hiệu năng
45	src/training/class_weight	Điều chỉnh số lớp	ML	Dữ liệu imbalance xử lý kém
46	.github/workflows/ci.yaml	Tự động hóa CI	Tech Lead + Backend	Không có guardrail tích hợp
47	mlruns/	Lịch sử thực nghiệm cục bộ	ML	Audit trail giảm
48	models/	Artifacts runtime thật	AI + ML	Runtime không thể nạp model
49	data/	Dữ liệu và output pipeline	ML	Không tái tạo được model
50	docs/handbook/*	Tài liệu handover hiện tại	Tech Lead + Solution	Kiến thức tổ chức không được truyền đạt

5.2 Ghi chú mức độ quan trọng

Các file trong top 10 có thể làm hệ thống ngừng hoạt động nếu bị xóa hoặc sửa sai. Các file từ hạng 11 đến 20 chi phối khả năng tái lập training và chất lượng mô hình. Các file từ 21 đến 36 quyết định khả năng dựng môi trường, tích hợp giao diện, và giám sát vận hành. Phần còn lại là tài sản điều phối, kiểm soát thay đổi, và tri thức tổ chức.

6 Phần 6 - Trace thực thi

6.1 Khởi động ứng dụng

File / hàm	Đầu vào	Đầu ra	Luồng lỗi
src/main.py app startup	env vars, model path	FastAPI app với state đã nạp	model load lỗi sẽ fail startup hoặc degrade theo path
src/model/config.py	params cấu hình	label map, supported languages	config thiếu gây runtime mismatch

<code>src/model/onnx_infer</code>	<code>ONNX model path, tokenizer path</code>	<code>inference object</code>	artifact thiếu hoặc không tương thích
<code>init</code>			

Chủ sở hữu: Backend cho orchestration, AI cho model load, Solution cho container và mount artifact.

6.2 Health Check

Route health nằm trong `src/main.py`. Nó trả trạng thái ứng dụng, mode đang chạy, và danh sách ngôn ngữ hỗ trợ. Output hiện phản ánh cấu hình runtime thật tốt hơn README.

6.3 Predict

Luồng predict:

1. UI gọi endpoint predict với văn bản.
2. Backend validate request model.
3. Runtime chọn model path và chạy tokenization.
4. ONNX Runtime trả logits.
5. Logic hậu xử lý chuyển logits sang probability và label.
6. API đóng gói response JSON.

Exception path chính: input rỗng, artifact không nạp được, tokenizer mismatch, lỗi ONNX Runtime, hoặc response serialization lỗi.

6.4 Explain

Explain path gọi thêm logic SHAP và/hoặc thông tin hỗ trợ giải thích. Đây là path nặng hơn predict thường. Chi phí khởi tạo và xử lý SHAP có thể làm độ trễ tăng mạnh.

6.5 Batch Predict

Batch path đọc CSV, validate cột văn bản, lặp qua các hàng và chạy dự đoán. Đây là xử lý đồng bộ trong request-response hiện tại. Endpoint `/batch_status/{job_id}` tồn tại nhưng không có job orchestration thực sự.

6.6 Evaluation

Route evaluate trong runtime có thể kích hoạt logic đánh giá. Đây là bề mặt admin nhưng hiện chưa có auth. Về vận hành, đây là rủi ro rõ ràng.

6.7 Training

Training được điều phối bởi DVC và script dưới `src/scripts`. Input là dữ liệu đã tiền xử lý và params; output là checkpoint, metric, và artifact theo dõi trong MLflow.

6.8 Export ONNX

Export path merge adapter với base model rồi sinh artifact ONNX để runtime nạp. Nếu bước này lỗi hoặc artifact không khớp tokenizer/config, runtime sẽ không phục vụ đúng.

6.9 Sequence diagram

```

1 sequenceDiagram
2     participant Boot as Startup
3     participant API as FastAPI
4     participant CFG as ModelConfig
5     participant ORT as ONNXInferenceModel
6
7     Boot->>API: khởi tạo app
8     API->>CFG: đọc cấu hình model
9     API->>ORT: load tokenizer và ONNX session
10    ORT-->>API: model ready

```

```

1 sequenceDiagram
2     participant FE as Frontend
3     participant API as /predict
4     participant M as Inference
5
6     FE->>API: text
7     API->>M: preprocess + infer
8     M-->>API: label + score
9     API-->>FE: JSON response

```

7 Phần 7 - Sổ tay Frontend

7.1 Owner

Nguyen Le Hong Nhi là chủ sở hữu chính của frontend Angular và trải nghiệm người dùng.

7.2 Kiến trúc

Frontend nằm trong app/sentiment-analysis-chatbot. Kiến trúc dùng Angular component-service truyền thống, không có state management phức tạp. Trạng thái chủ yếu nằm trong component tree và service gọi API.

7.3 Components

Các component đáng chú ý:

- `app.component.ts`: app shell, orchestration chat flow
- `chat-input.component.ts`: nhập câu, chọn ngôn ngữ, gửi request
- `batch-upload.component.ts`: upload CSV để batch predict

7.4 Services và state flow

`sentiment-analysis.service.ts` là đầu mối API integration. Service này đang là nơi drift xuất hiện: frontend có lựa chọn ngôn ngữ nhưng request predict thường chỉ gửi `{text}` mà không đẩy lang xuống backend.

7.5 Explainability UI

UI có khả năng hiển thị giải thích để tăng niềm tin người dùng. Tuy nhiên chi tiết giải thích phụ thuộc hoàn toàn vào payload backend; frontend không tự tính gì.

7.6 Batch Upload UI

Batch UI phục vụ người dùng tải tệp CSV. Vì backend batch chạy đồng bộ, trải nghiệm người dùng phụ thuộc trực tiếp vào thời gian request; không có polling job thật.

7.7 Rủi ro chính

- Drift giữa option ngôn ngữ UI và ngôn ngữ backend thật.
- Khả năng UI hứa hẹn async behavior nhiều hơn backend cung cấp.
- Tăng blast radius nếu thay đổi response shape mà không cập nhật model frontend.

7.8 Điểm mở rộng

- đồng bộ contract lang với backend
- thêm progress state đúng với batch runtime thật
- tách rõ các chế độ predict nhanh và explain nặng

8 Phần 8 - Sổ tay Backend

8.1 Owner

Pham Duc Long là chủ sở hữu chính của FastAPI runtime và API contract.

8.2 Cấu trúc FastAPI

Backend tập trung phần lớn trong `src/main.py`. Đây là dấu hiệu một runtime còn tập trung logic ở một nơi, chưa tách route layer, service layer, và admin surface thành module độc lập.

8.3 Contracts và validation

Backend dùng request/response model để validate cơ bản. Tuy vậy, drift giữa `contracts/README.md`, OpenAPI docs tham chiếu, và route thật vẫn tồn tại. Runtime thật hiện dùng các path gốc như `/predict`, `/health`, trong khi frontend-facing path đi qua Nginx với prefix `/api/`.

8.4 Bản đồ ownership API

API	Mục đích	Owner
/health	báo trạng thái runtime và khả năng model	Backend
/predict	sentiment một câu	Backend + AI
/explain	dự đoán có giải thích	Backend + AI
/batch_predict	chăm CSV đồng bộ	Backend + AI
/batch_status/{job_id}	status mock	Backend
/evaluate	kích hoạt đánh giá	Backend + ML
/metrics	expose Prometheus metrics	Backend + Solution

8.5 Xử lý lỗi

Lỗi hiện được xử lý ở mức route và runtime, nhưng chưa có error taxonomy mạnh, chưa có auth failure path, và chưa có idempotent job control. Đối với vận hành, điều này có nghĩa là incident sẽ phải đọc log và tái hiện thủ công nhiều hơn.

8.6 Khoảng trống backend

- Not Implemented: authentication
- Not Implemented: authorization
- Not Implemented: durable batch job system
- Not Implemented: persistent request/job state store
- Partial: contract alignment giữa docs, frontend, và runtime

9 Phần 9 - Sổ tay AI inference

9.1 Owner

Do Quoc Trung là chủ sở hữu chính của inference pipeline.

9.2 Pipeline

Pipeline inference đi từ text sang tensor, logits, probability, rồi label. Runtime mặc định dùng ONNX mode, vì vậy hiệu năng và ổn định phụ thuộc lớn vào chất lượng export artifact.

9.3 Tokenization và tensor

Tokenizer chuyển text thành token ids, attention mask và các tensor đầu vào tương thích mô hình. Nếu tokenizer và ONNX artifact không cùng phiên bản logic, kết quả có thể sai lệch âm thầm.

9.4 ONNX Runtime

ONNX Runtime là động cơ serving chính. Đây là điểm tốt về hiệu năng, nhưng cũng là nơi coupling chặt với artifact export và shape mong đợi của mô hình.

9.5 Logic prediction

Sau khi có logits, pipeline tính probability và map sang label. Đây là chỗ logic nghiệp vụ và kỹ thuật gặp nhau: thử người dùng thấy chỉ là nhãn và confidence, nhưng chất lượng thực nằm ở calibration và evaluation phía sau.

9.6 ABSA

ABSA hiện dùng zero-shot classifier từ Hugging Face. Điều này hữu ích cho demo nhưng chi phí tính toán lớn hơn inference sentiment chính, và độ ổn định phụ thuộc model phụ trợ bên ngoài.

9.7 SHAP

SHAP được dùng cho explainability. Đây là tính năng có giá trị demo và giải thích, nhưng là nguồn chi phí độ trễ đáng kể. Nếu chạy đồng bộ trên request online, nó có thể gây spike latency.

9.8 Phát hiện ngôn ngữ

Language detection có hiện diện trong codebase, nhưng năng lực runtime thực tế vẫn gói quanh tập ngôn ngữ hỗ trợ từ config. Hệ thống thật hiện hỗ trợ `en` và `vi`.

9.9 Phân tích độ trễ

- Predict sentiment thuần là path nhanh nhất.
- Batch predict mở rộng gần tuyến tính theo số dòng nếu không có song song hóa backend.
- Explain path đắt hơn do SHAP.
- ABSA cũng đắt hơn predict thường vì dùng model phụ.

Các test hiệu năng trong `tests/perf` và `tests/model` đóng vai trò guardrail nhưng chưa thay thế được benchmarking trên hạ tầng production thật.

10 Phần 10 - Sổ tay ML

10.1 Owner

Duong Binh An là chủ sở hữu chính của vòng đời training và dữ liệu.

10.2 Chiến lược dữ liệu

Repo chứa raw/processed data và pipeline xử lý qua `src/data`. Chất lượng mô hình phụ thuộc nặng vào validation dữ liệu hơn là vào vài thay đổi nhỏ trong serving.

10.3 DVC

`dvc.yaml` điều phối pipeline download, preprocess, validate, train, evaluate. Đây là phần quan trọng để tái lập quy trình huấn luyện và để chủ dự án tương lai kiểm toán được đường đi của dữ liệu.

10.4 MLflow

MLflow được dùng để ghi experiment, metric, và trace huấn luyện. Trong Compose, MLflow cũng được dựng như một service riêng.

10.5 LoRA và PEFT

Huấn luyện dùng LoRA/PEFT để tinh chỉnh hiệu quả hơn so với full fine-tuning. Đây là lựa chọn hợp lý cho MVP/Advanced MVP vì tiết kiệm chi phí và giảm kích thước thay đổi cần quản lý.

10.6 Đánh giá và fairness

Codebase có đánh giá metric nhưng fairness chưa thấy thành năng lực vận hành hoàn chỉnh. Vì vậy phải ghi rõ: fairness governance ở mức **Partial**, chưa phải chương trình kiểm soát sản phẩm hoàn chỉnh.

10.7 Vòng đời training

```
1 flowchart LR
2   Download --> Preprocess
3   Preprocess --> Validate
4   Validate --> Finetune
5   Finetune --> Evaluate
6   Evaluate --> MLflow
7   Finetune --> ExportONNX
```

10.8 Khoảng trống ML

- Not Implemented: closed-loop feedback ingestion từ production
- Partial: fairness governance
- Partial: artifact promotion governance giữa train và runtime
- Partial: reproducible environment governance ngoài local/compose scope

11 Phần 11 - Sổ tay Solution Engineering

11.1 Owner

Duong Hong Quan là chủ sở hữu chính cho tích hợp end-to-end, kiến trúc triển khai, và readiness vận hành.

11.2 Tích hợp end-to-end

Giá trị của vai trò Solution trong repo này là nối các phần rời rạc thành một chuỗi vận hành thống nhất: UI, proxy, API, model artifact, monitoring, training lineage.

11.3 Kiến trúc triển khai

Compose là bề mặt triển khai chính. Nó phù hợp cho tích hợp cục bộ, demo, và môi trường pilot nhỏ; chưa thể xem là chiến lược production scale hoặc rollout governance đầy đủ.

11.4 Chiến lược monitoring

Prometheus scrape backend metrics và Grafana hiển thị dashboard. Alert rules đã có nhưng sức mạnh thực tế phụ thuộc việc đội ngũ có playbook và ownership phản ứng hay không.

11.5 Đánh giá readiness vận hành

Kiến trúc đủ rõ để vận hành demo và pilot, nhưng còn thiếu:

- auth và admin boundary
- durable state cho batch hoặc evaluate workflow
- secret management chặt chẽ
- deployment promotion có kiểm soát

12 Phần 12 - Ma trận ownership

12.1 Ma trận ownership thư mục

Thư mục	Mục đích	Owner chính	Owner phụ
app/sentiment-analysis-chatbot	UI runtime	Nguyen Le Hong Nhi	Duong Hong Quan
src/main.py và API runtime	orchestration backend	Pham Duc Long	Do Quoc Trung
src/model	inference và export	Do Quoc Trung	Duong Binh An
src/data	data prep và validation	Duong Binh An	Le Quang Tuyen
src/training	training internals	Duong Binh An	Do Quoc Trung
src/scripts	train/eval/export entrypoints	Duong Binh An	Do Quoc Trung
infra	monitoring và provisioning	Duong Hong Quan	Le Quang Tuyen
contracts	API alignment	Pham Duc Long	Duong Hong Quan
docs	tri thức tổ chức	Le Quang Tuyen	Duong Hong Quan
tests	regression và guardrails	Pham Duc Long	Do Quoc Trung

12.2 Ma trận ownership service

Service	Phạm vi	Owner
Angular frontend	UX, state giao diện, API client	UI/UX
Nginx proxy	routing frontend tới backend	Solution
FastAPI backend	request handling, metrics, orchestration	Backend
ONNX inference	model serving path	AI
Training pipeline	fine-tuning và evaluation	ML
Prometheus/Grafana	observability	Solution
Release coordination	thay đổi liên team	Tech Lead

12.3 Operational, security, documentation, knowledge ownership

- Operational ownership: Solution dẫn đầu, Backend và AI hỗ trợ theo symptom.
- Security ownership: Tech Lead chịu trách nhiệm quyết định ưu tiên; Backend và Solution sửa implementation; AI/ML sở hữu model artifact hygiene.
- Documentation ownership: Tech Lead sở hữu tài liệu hệ thống; từng lead sở hữu handbook phần mình.
- Knowledge ownership: hiện chưa cân bằng hoàn toàn; cần giảm phụ thuộc vào cá nhân có nhiều ngữ cảnh nhất.

13 Phần 13 - Ma trận RACI

Hoạt động	Tech Lead	ML	AI	Solution	Backend	UI/UX
Feature development	A	C	C	C	R	R
Model training	C	A/R	C	I	I	I
Model deployment	A	C	R	R	C	I
API changes	C	I	C	C	A/R	I
Production release	A	C	C	R	R	C
Monitoring	C	I	C	A/R	R	I
Security	A	C	C	R	R	I
Incident response	A	I	R	R	R	I
Architecture changes	A/R	C	C	C	C	C

14 Phần 14 - Data và model lineage

Giai đoạn	Producer	Consumer	Owner	Artifact / vòng đời
Raw data	downloader và nguồn dữ liệu	preprocessing	ML	dữ liệu nguồn thô
Validation	validators	training pipeline	ML	báo cáo chất lượng dữ liệu
Preprocessing	pipeline	finetuning	ML	dataset đã chuẩn hóa
Training	HF trainer + PEFT	evaluation, export	ML	checkpoint adapter và metric
Evaluation	evaluate scripts	MLflow, release decision	ML	metric và confusion outputs
MLflow	training/eval call-backs	con người và governance	ML	lịch sử thí nghiệm
Export	onnx exporter	runtime backend	AI + ML	ONNX model, tokenizer, config
Runtime load	FastAPI startup	predict/explain requests	AI + Backend	model session trong bộ nhớ
Prediction	runtime pipeline	UI và caller	AI + Backend	label, probability, explain payload

15 Phần 15 - Security review

Vấn đề	Mức độ	Owner	Mitigation
Không có authentication cho API	Critical	Backend Solution	+ thêm authn, token/session policy, bảo vệ admin endpoints
Không có authorization cho evaluate/admin-like actions	Critical	Backend Solution	+ tách admin surface và policy kiểm soát quyền
CORS mở rộng	High	Backend	giới hạn origin theo môi trường
Grafana mặc định admin/admin	High	Solution	đổi secret, externalize credential, rotate ngay

Secrets quản lý còn sơ khai	High	Solution + Tech Lead	dùng secret store hoặc env governance tốt hơn
Batch upload surface cần kiểm soát file	Medium	Backend	tăng validate MIME, kích thước, schema CSV
Supply-chain risk từ dependency ML và UI	Medium	Tech Lead + Solution	pin version và quét CVE định kỳ
Model artifact integrity chưa có ký/phê duyệt rõ	Medium	AI + ML	thiết lập promotion và checksum governance
Public evaluate endpoint có thể bị lạm dụng	High	Backend	hạn chế truy cập hoặc bỏ khỏi runtime công khai

16 Phần 16 - Performance review

16.1 Startup Time

Startup time chịu chi phối chủ yếu bởi việc nạp tokenizer và ONNX session. Đây là chi phí chấp nhận được cho service đơn lẻ nhưng cần đo lại nếu chuyển sang autoscaling.

16.2 Memory Usage

Memory footprint phụ thuộc artifact model, tokenizer, và các thành phần giải thích. SHAP và ABSA là hai nguồn làm tăng footprint khi bật đầy đủ.

16.3 CPU Usage

Predict sentiment thường là CPU-friendly hơn explain path. Batch path đồng bộ sẽ làm CPU tăng tuyến tính theo số mẫu nếu không có hàng đợi nền.

16.4 Inference Latency

Latency thấp nhất ở predict sentiment đơn thuần. Explain và ABSA là các path đắt hơn rõ rệt. Test hiệu năng có tồn tại nhưng chưa tương đương sản xuất thật.

16.5 Batch Throughput

Theo test benchmark, batch sentiment-only có guardrail throughput. Tuy vậy backend hiện chưa có queue hay worker layer riêng nên throughput sẽ bị giới hạn bởi request handler và tài nguyên container đơn.

16.6 SHAP Cost

SHAP là tính năng đắt nhất về độ trễ trong inference-facing features. Nếu cần scale, nên tách explain thành job hoặc endpoint riêng với SLA khác.

16.7 ABSA Cost

ABSA dựa vào zero-shot classifier nên không rẻ. Nó phù hợp làm capability nâng cao hơn là path mặc định cho mọi request.

16.8 Giới hạn mở rộng

- không có durable asynchronous execution
- không có cache chiến lược cho explain/ABSA
- app state và orchestration tập trung trong một process
- rollout model chưa có version routing hoặc canary

17 Phần 17 - Sổ tay vận hành

Sự cố	Triệu chứng	Nguyên nhân gốc	Bước debug	Hướng xử lý / owner
API Failure	/health lỗi, UI không trả kết quả	container chết, model load fail, route lỗi	xem log backend, kiểm tra mount model, gọi health trực tiếp	Backend + Solution khôi phục container và config
Model Failure	predict 500 hoặc kết quả vô nghĩa	artifact lỗi, tokenizer mismatch, export hỏng	kiểm tra artifact path, log startup, chạy benchmark cục bộ	AI + ML thay artifact hoặc export lại
Latency Spike	request chậm, timeout batch/explain	SHAP, ABSA, CPU contention, batch lớn	so sánh predict thường với explain, xem metrics latency	AI + Backend giảm tải, giới hạn path nặng
Monitoring Failure	Grafana trống, Prometheus không scrape	config sai, container down, target name đổi	kiểm tra Prometheus targets và datasource	Solution sửa provisioning
Deployment Failure	Compose không dựng đủ stack	image build fail, port clash, env drift	kiểm tra Docker logs, file Compose, env	Solution xử lý build/network
Batch Failure	upload CSV lỗi hoặc status không nhất quán	schema CSV sai, logic sync timeout, status mock	kiểm tra payload, kích thước file, log route batch	Backend sửa validation hoặc chuyển async thật
Data Pipeline Failure	DVC stage fail, train không chạy	thiếu data, params lỗi, dependency drift	chạy từng stage, đọc output validator, xem MLflow	ML khắc phục data hoặc config

18 Phần 18 - Sổ đăng ký technical debt

Mức	Mô tả	Tác động	Owner	Khuyến nghị
Critical	Không có auth/authz	không thể mở production an toàn	Backend + Solution	thêm auth, tách admin surface, kiểm soát quyền
Critical	Batch status là mock trong khi batch chạy sync	giao diện và kỳ vọng tích hợp bị lệch	Backend	hoặc bỏ endpoint status, hoặc triển khai job system thật

High	Drift giữa UI, docs, và back-end về lang và route shape	bug tích hợp, hiểu sai phạm vi sản phẩm	UI/UX Backend Solution	+	chuẩn hóa contract và test E2E
High	Grafana dùng credential mặc định	rủi ro truy cập trái phép	Solution		đổi secret ngay
High	/evaluate công khai	bề mặt lạm dụng và chi phí vận hành	Backend		chuyển thành admin-only hoặc remove khỏi public runtime
Medium	Logic runtime tập trung nhiều trong <code>src/main.py</code>	khó bảo trì dài hạn	Backend		tách router/service/module boundary
Medium	Chưa có persistent store cho job và feedback	không thể audit/khôi phục tốt	Backend Solution	+	thêm DB/queue phù hợp
Medium	Fairness governance chưa hoàn chỉnh	khó mở rộng enterprise	ML		thêm báo cáo fairness vào release gate
Low	README và contract docs cũ hơn implementation	onboarding và due diligence chậm hơn	Tech Lead		cập nhật tài liệu theo code

19 Phần 19 - Đánh giá tổ chức

19.1 Cấu trúc đội ngũ

Cấu trúc vai trò hiện hợp lý cho một dự án AI ứng dụng: Tech Lead điều phối, ML chịu dữ liệu/training, AI chịu inference/model serving, Backend chịu API, Solution chịu tích hợp và deploy, UI/UX chịu frontend.

19.2 Phân phối tri thức và bus factor

Bus factor hiện chưa cao. Nhiều quyết định quan trọng nằm ở giao điểm giữa AI, ML, và Solution. Nếu một người rời đi mà không có handbook này hoặc không có runbook chi tiết, thời gian phục hồi kiến thức sẽ đáng kể.

19.3 Khoảng trống ownership

- ownership của admin/security surface chưa được thể hiện trong code boundary
- ownership của hợp đồng tích hợp FE-BE chưa có guardrail tự động đủ mạnh
- ownership của artifact promotion từ training sang runtime còn pha trộn giữa AI và ML

19.4 Khuyến nghị

- thiết lập release gate liên team với checklist auth, contract, benchmark, dashboard
- tạo owner rõ cho model promotion và rollback
- yêu cầu mỗi lead duy trì runbook và test tương ứng phạm vi mình sở hữu

20 Phần 20 - Scorecard production readiness

Hạng mục	Điểm	Nhận xét
Architecture	72	Có cấu trúc đủ rõ, có online/offline split, nhưng boundary chưa đủ chặt.
Security	28	Thiếu auth/authz và có secret hygiene yếu.
Reliability	54	Có health, metrics, containerization, nhưng state và batch orchestration còn yếu.
Performance	67	ONNX là nền tốt; explain và ABSA vẫn đắt.
Testing	63	Có API và perf tests, nhưng chưa phủ hết contract drift và security.
Monitoring	70	Có Prometheus/Grafana/alerts, nhưng playbook và hardening chưa hoàn thiện.
ML Lifecycle	78	DVC, MLflow, PEFT, export ONNX đã thành chuỗi khá tốt.
Deployment	58	Compose tốt cho tích hợp, chưa đủ cho production-grade rollout.
Operations	52	Có tín hiệu observability nhưng thiếu nhiều control production.
Documentation	74	Sau handbook này, tri thức rõ hơn; docs cũ vẫn còn drift.
Ownership	73	Vai trò rõ, nhưng cần siết owner ở contract và release gates.

Điểm readiness tổng hợp có trọng số: 62/100. Kết luận: đủ mạnh cho Advanced MVP và pilot kiểm soát chặt; chưa nên xem là production-ready cho enterprise.

21 Phần 21 - Roadmap

21.1 Roadmap 3 tháng

- Triển khai authentication, authorization, và tách admin endpoints. Owner: Backend + Solution. Priority: P1.
- Chuẩn hóa contract giữa frontend, backend và docs, đặc biệt cho lang và batch behavior. Owner: UI/UX + Backend + Solution. Priority: P1.
- Thay credential mặc định, siết secret management, và cập nhật runbook bảo mật. Owner: Solution. Priority: P1.

21.2 Roadmap 6 tháng

- Chuyển batch sang worker/job model có trạng thái bền. Owner: Backend + Solution. Priority: P1.
- Thiết lập artifact promotion, rollback, và checksum governance cho model. Owner: AI + ML. Priority: P1.
- Mở rộng test E2E và release gates cho latency, contract, security. Owner: Tech Lead + Backend + AI. Priority: P2.

21.3 Roadmap 12 tháng

- Nâng cấp từ Compose sang chiến lược triển khai có secret handling và rollout control tốt hơn. Owner: Solution. Priority: P1.
- Tách SLA giữa predict nhanh và explain/ABSA path nặng. Owner: Backend + AI. Priority: P2.
- Chỉ mở rộng đa ngôn ngữ sau khi data và evaluation parity được chứng minh. Owner: ML + AI + UI/UX. Priority: P2.

22 Phần 22 - Hướng dẫn onboarding

Vai trò / mốc thời gian	Kế hoạch học
Tech Lead Day 1	Đọc <code>src/main.py</code> , <code>docker-compose.yml</code> , <code>dvc.yaml</code> , handbook này, và kiểm tra discrepancy list.
Tech Lead Day 3	Chạy toàn bộ stack local, xác nhận runtime path, monitoring path, training path.
Tech Lead Day 7	Review release gate, security debt, ownership matrix.
Tech Lead Day 14	Dẫn một mini readiness review nội bộ.
Tech Lead Day 30	Có thể quyết định ưu tiên kiến trúc, bảo mật, và phát hành.
ML Engineer Day 1	Đọc <code>src/data/</code> , <code>src/training/</code> , <code>params.yaml</code> , <code>dvc.yaml</code> .
ML Engineer Day 3	Chạy download, preprocess, validate.
ML Engineer Day 7	Chạy finetune và evaluate một cấu hình nhỏ.
ML Engineer Day 14	Theo dõi MLflow và xác nhận output export.
ML Engineer Day 30	Tự vận hành toàn bộ lifecycle training.
AI Engineer Day 1	Đọc <code>src/model/</code> , benchmark tests, và startup load path.
AI Engineer Day 3	Chạy predict, explain, benchmark ONNX.
AI Engineer Day 7	Xác minh tokenizer, ONNX session, latency path.
AI Engineer Day 14	Điều tra một regression inference giả lập.
AI Engineer Day 30	Tự xử lý model load, ONNX, SHAP, ABSA, latency.
Backend Engineer Day 1	Đọc <code>src/main.py</code> , <code>contracts/</code> , <code>tests/test_api.py</code> .
Backend Engineer Day 3	Chạy API local và gọi các endpoint chính.
Backend Engineer Day 7	So sánh route thật với docs và frontend.
Backend Engineer Day 14	Sửa một thay đổi contract nhỏ kèm test.
Backend Engineer Day 30	Sở hữu route changes, validation, metrics, failure handling.
Solution Engineer Day 1	Đọc Dockerfile, Compose, Nginx, Prometheus, Grafana config.
Solution Engineer Day 3	Dựng stack đầy đủ.
Solution Engineer Day 7	Kiểm tra dashboard, alert rules, và secret posture.
Solution Engineer Day 14	Tập xử lý sự cố monitoring hoặc deployment.
Solution Engineer Day 30	Dẫn vận hành môi trường tích hợp và readiness review.
UI/UX Engineer Day 1	Đọc app shell, service, components và contract models.
UI/UX Engineer Day 3	Chạy frontend và test luồng chat/batch.
UI/UX Engineer Day 7	Kiểm tra language selector và response mapping.
UI/UX Engineer Day 14	Sửa một thay đổi payload hiển thị.
UI/UX Engineer Day 30	Sở hữu chat flow, batch UX, explain UX, và contract alignment.

23 Phần 23 - CTO review

23.1 Có phê duyệt production launch không?

Không, chưa cho production mở hoặc production enterprise. Có thể phê duyệt cho demo nội bộ và pilot kiểm soát chặt sau khi xử lý các lỗ hổng bảo mật cấp cao.

23.2 Có phê duyệt enterprise customers không?

Không. Mức kiểm soát bảo mật, durability, và release governance hiện chưa đủ.

23.3 Có phê duyệt scaling không?

Chưa. Hướng kiến trúc có thể mở rộng, nhưng mô hình vận hành và job execution hiện chưa sẵn sàng.

23.4 Điều đáng lo nhất là gì?

Điều đáng lo nhất là cảm giác trưởng thành bề ngoài có thể tạo ra tự tin sai. Repository nhìn khá đầy đủ, nhưng nhiều kiểm soát nền tảng còn thiếu hoặc mới ở mức một phần.

23.5 Khoản đầu tư đầu tiên nên là gì?

Đầu tư đầu tiên nên là hardening nền tảng: authentication, authorization, quản lý admin surface, governance artifact model, và release gates liên team.

Discrepancy implementation

- README health example chỉ hiện ["en"]; implementation thật hỗ trợ en và vi.
- README mô tả batch giống một async job; implementation hiện là request-response đồng bộ và endpoint status là mock.
- Frontend hiển thị nhiều lựa chọn ngôn ngữ; backend thực tế hỗ trợ en và vi; request predict từ frontend thường không gửi lang tường minh.
- Compose dùng MLflow URL nội bộ `http://mlflow:5000`, host publish 5005, trong khi tham số mặc định có chỗ giả định `http://localhost:5000`; giả định môi trường chưa thống nhất.
- `contracts/README.md` dùng ví dụ `/api/v1/*`; implementation thật dùng các route gốc như `/predict`, `/health`, và Nginx thêm prefix `/api/` ở bề mặt frontend.